Assignment 1

ADSA

Manny Asbanu
a1926920
Repository: https://github.com/mannyasbanu/ADSA

Description

For the task I wrote implementations in C++ of the School Method for integer addition, Karatsuba Algorithm for integer multiplication, and School Method for integer division.

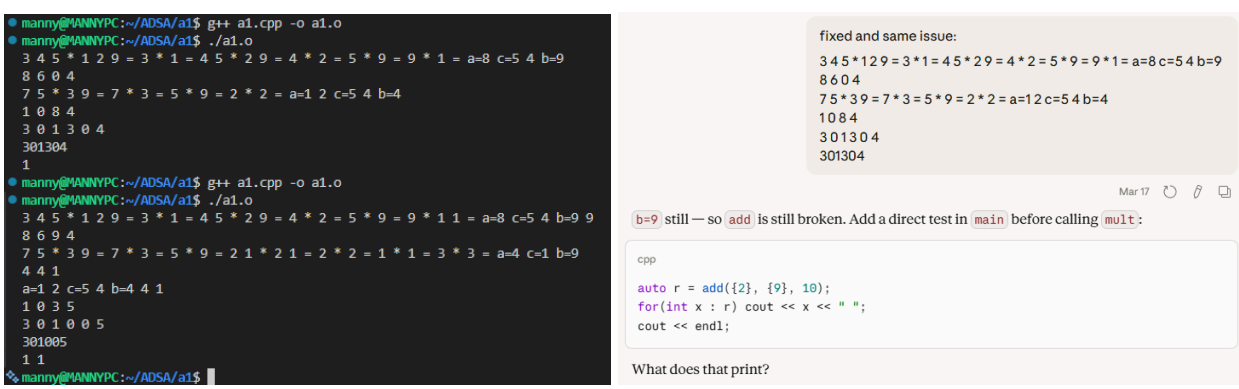
To support my implementations, I also created helper functions Trim: to remove trailing zeroes, Sub: to subtract integers, Greater: for greater than comparison between integers, and Equal: for equality comparison between integers.

Each implementation was designed and mostly implemented without the use of AI. Youtube videos and online resources were used for assistance with syntax and theory. All functions operate on a vector representing an integer, both input and output of each function assumes the integer is in reverse order. Test case input is manipulated in main.

Use of AI

AI was used to assist in debugging errors and small logic issues within my implementations. The typical workflow would involve attempting to debug myself, and then passing the output error and function to Claude Code if I remained stuck.

For logical errors, I would assist Claude in debugging by inserting print statements within the function for it to assess the output along different steps. Below is an example of how this would look, where it located the logical error in my Karatsuba algorithm to be from my Add function. After overlooking, I found an error in the Add function and Karatsuba worked after.



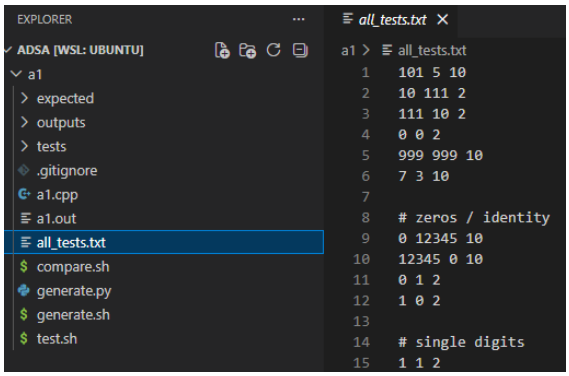
Claude would often open my eyes to tiny logical bugs or errors that I had overlooked, and likely would have spent ages trying to find.

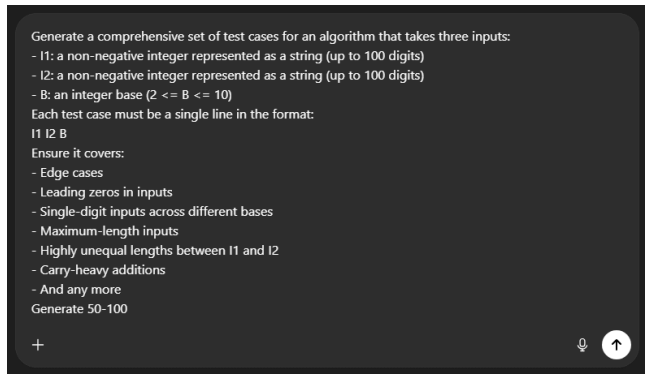
Testing Process

ChatGPT assisted with the testing process, enabling me to conduct large scale batch testing efficiently. We developed a clear testing workflow described below.

1. Test Case Generation

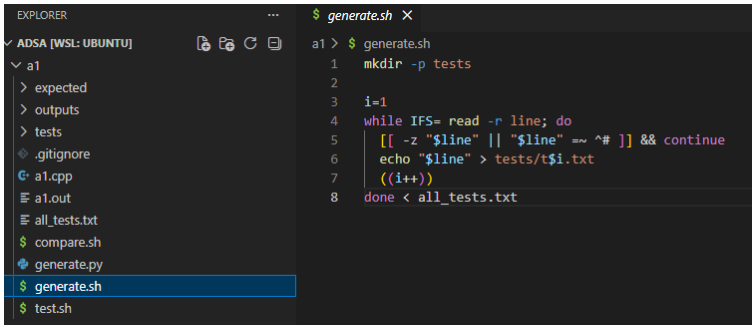
Using ChatGPT, I generated an extensive list of labeled test cases copy pasted into a single text file. These were generated by providing GPT with the algorithm and problem descriptions and example test case inputs and outputs. GPT would automatically consider edge cases, positive testing, corner cases, stress testing, etc.

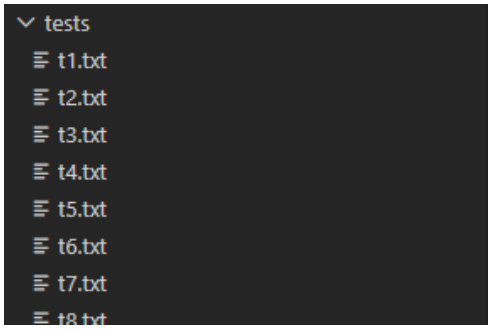




2. Test Case Files Creation

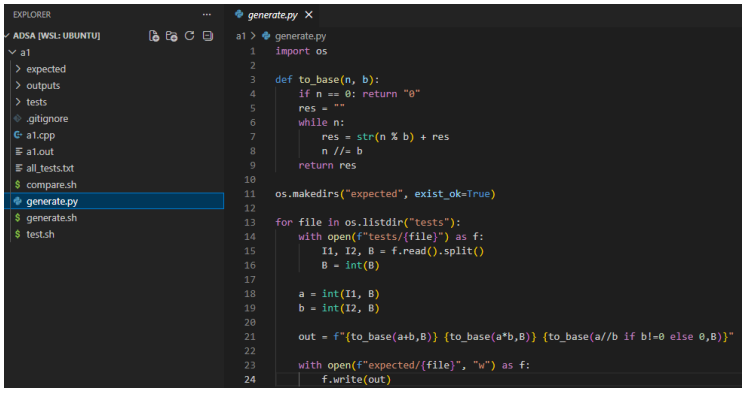
Using a batch script, the test cases pasted into the text file were automatically separated into individual labeled test case text files. This matches the format of each test case consisting of a single line. Removing the manual labour of creating each individually.





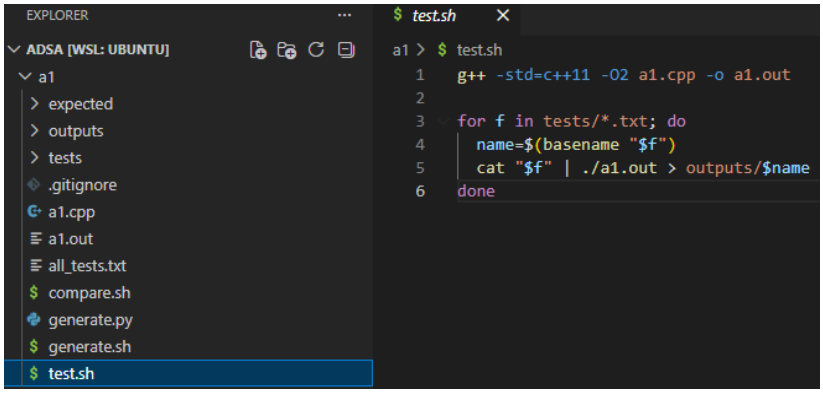
3. Expected Output Generation

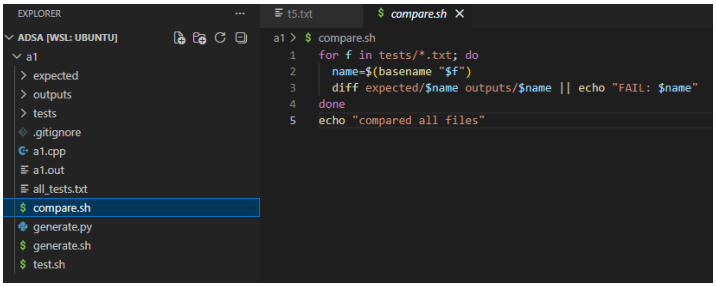
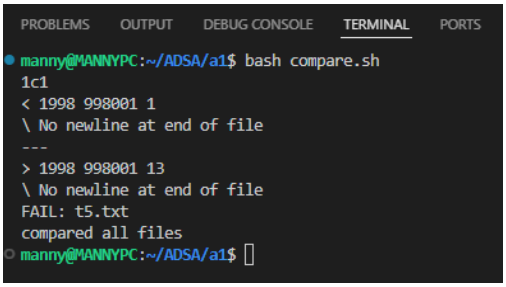
Using a python program, I efficiently generated expected outputs for each text file in its own output file. The expected output files and test files were named the same but located in different directories. This would help with comparison later.



4. Testing

Using a batch script, I automatically compiled a1.cpp and ran it on every test case file. The output files were passed into another directory, each with the same name as the relevant test case file. This would help with comparison later.



5. Verifying Output

Using a batch script, I automated the comparison process. This automatically compares the contents of all files inside the output directory to those in the expected directory. Any time the content differed, it would be listed in the terminal alongside the number of the relevant test case. This way I could locate exactly which test cases failed.